Démêlo — Ma partie Supabase

Oral du 29 octobre 2026 · Prise de parole : environ 1 minute · Sujet : architecture des données, rôles et sécurité RLS

Mon discours (≈ 1 minute)

> Pour Démêlo, nous avons choisi **Supabase** : une vraie base PostgreSQL, avec authentification et API intégrées.
>
> Elle repose sur **quatre tables reliées par des clés étrangères** : les **sources**, les 15 organismes officiels comme l'INSEE ou le Sénat ; les **contenus**, les articles des médias ; les **affirmations**, les phrases que nous vérifions ; et les **preuves**, qui relient chaque affirmation à un document officiel.
>
> Aujourd'hui : **760 contenus** — 260 vrais articles et 500 lignes fictives de test —, **10 affirmations** et **13 preuves**.
>
> Comme la clé du site est publique, nous avons activé la **Row Level Security** sur les quatre tables, avec **12 règles** écrites par nous. Un **visiteur** lit seulement les analyses publiées : il ne voit ni les données fictives, ni les analyses en cours. Seul un **administrateur**, reconnu par son rôle, peut écrire, et des **triggers** contrôlent chaque publication.
>
> Résultat : **52 tests SQL, 52 réussis**.

Environ 150 mots : 55 à 65 secondes à un rythme posé.

Les 6 chiffres à retenir

4	760	15	10	13	52/52
tables	contenus (260 réels + 500 fictifs)	sources	affirmations (9 publiées + 1 en revue)	preuves vérifiées	tests SQL réussis

Le fil conducteur en 5 mots

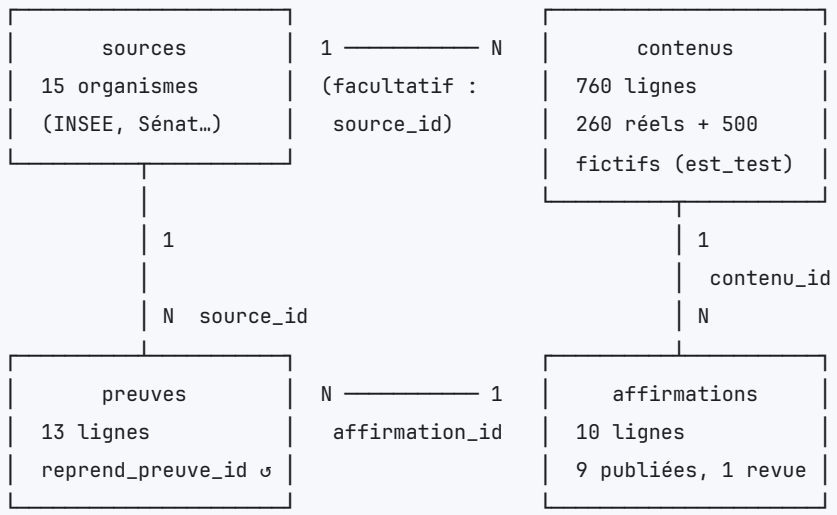
Supabase → 4 tables → relations → RLS → tests.

1. Architecture des données

1.1 Pourquoi Supabase ?

- **PostgreSQL complet** : clés étrangères, contraintes, vues, fonctions et triggers, et non un simple tableur.
- **Authentification intégrée (Supabase Auth)** : les comptes, les mots de passe et les jetons de connexion (JWT) sont gérés pour nous.
- **API automatique (PostgREST)** : le site lit les données directement, avec une **clé publique** (*publishable*).
- **Sécurité au niveau des lignes (RLS)** : indispensable, puisque la clé du site est publique.

1.2 Schéma des relations



+ 3 clés vers auth.users (traçabilité) : affirmations.created_by, affirmations.reviewed_by, preuves.ajoutee_par

8 clés étrangères au total : 5 entre nos tables, plus 3 vers `auth.users`.

1.3 Les quatre tables

`sources` — les organismes de vérification

Objectif	La liste fermée des **15 organismes** dont les documents peuvent servir de preuve.
Clé primaire	`id bigint` (généré automatiquement)
Colonnes clés	`nom` (unique) · `type_source` : `institution`, `statistique`, `juridique` ou `fact_checking` · `url_base` · `perimetres text[]` (domaines autorisés, par exemple `senat.fr` ou `ec.europa.eu/eurostat`) · `reutilisation` : `ouverte` ou `lien_seulement` · `active`
Relations	Référencée par `contenus.source_id` (facultatif) et par `preuves.source_id` (obligatoire).
Exemple réel	**Sénat** (`id 19`), périmètre `senat.fr`, licence ouverte. **AFP Factuel** est en `lien_seulement` : aucun extrait n'est stocké, seulement une empreinte du passage.

`contenus` — les articles

Objectif	Les articles politiques collectés dans les flux RSS des médias, plus les données fictives de test.

Clé primaire	`id bigint`
Clé étrangère	`source_id` vers `sources(id)`, **facultative** : remplie seulement si l'article est publié par l'un des 15 organismes.
Colonnes clés	`titre` · `url_originale` (**unique**) · `media_nom` · `domaine_origine` (**colonne générée**, calculée à partir de l'URL) · `date_publication` · `theme` · `visible_public` · `est_test` · `lot_import` · `date_collecte`, `vu_le` · `statut_traitement`
Unicité	`url_originale`, et le couple (`domaine_origine`, `identifiant_externe`) : c'est ce qui empêche les doublons.
Exemple réel	`id 650` : 20 Minutes, « Colère sociale : Le Sénat adopte le projet de loi "casseurs-payeurs" », lot `rss-2026-10-28`. Les fictifs : lot `fictif-v1`, domaine `demelo-fictif.invalid`.

`affirmations` — les phrases vérifiées

Objectif	Une phrase précise tirée d'un article, son analyse et son statut de publication.
Clé primaire	`id bigint`
Clés étrangères	`contenu_id` vers `contenus(id)` (**obligatoire**, `ON DELETE RESTRICT`) · `created_by` et `reviewed_by` vers `auth.users(id)`
Colonnes clés	`texte_affirmation` (citation exacte, 10 à 400 caractères) · `citation_verifiee` · `nature_enonce` : `factuel`, `opinion`, `promesse ou prevision` · `statut_analyse` : `non_traite`, `en_cours`, `analyse_disponible`, `verification_insuffisante`, `revue_requise` ou `retire` · `verdict` : `vrai`, `faux`, `trompeur ou non_verifiable` · `explication` · `limites` · `motif_abstention` · `sensible` et `motif_sensibilite` · `recoupement jsonb` · `modele_ia` · `version` · `published_at` · `historique jsonb`
Exemple réel	`id 59` : « Le Sénat l'a adopté dans la nuit à 229 voix contre 105 » → **vrai**, publiée. `id 68` : cas sensible, en `revue_requise`.

`preuves` — les documents officiels

Objectif	Chaque document officiel réellement consulté, qu'il ait été retenu (vérifié) ou écarté (échec tracé).
Clé primaire	`id bigint`
Clés étrangères	`affirmation_id` vers `affirmations(id)` (`ON DELETE CASCADE`) · `source_id` vers `sources(id)` · `reprend_preuve_id` vers `preuves(id)` (une preuve qui en reprend une autre) · `ajoutee_par` vers `auth.users`
Colonnes clés	`url_preuve`, `url_finale`, `http_statut` · `empreinte_sha256` (empreinte du document) · `titre_document` · `date_document` ou `version_document` · `extrait ou passage_empreinte` · `passage_verifie` · `domaine_conforme` (calculé par le trigger) · `type_relation` : `confirme`, `contredit ou contextualise` · `nature` : `primaire` ou `secondaire` · `statut_preuve` : `verifiee`, `inaccessible` ou `rejetee` · `version_analyse`
Unicité	(`affirmation_id`, `version_analyse`, `url_preuve`)
Exemple réel	Le scrutin public **n° 41** du Sénat (`senat.fr/scrutin-public/2026/scr2026-41.html`), HTTP 200, passage « 229 pour 105 contre » retrouvé mot pour mot → `confirme`, `primaire`.

Les données actuelles

Table	Lignes	Détail
`sources`	15	toutes actives
`contenus`	760	260 réels (7 médias) + 500 fictifs

Table	Lignes	Détail
`affirmations`	10	9 `analyse_disponible`, toutes avec le verdict « vrai » · 1 `revue_requise`
`preuves`	13	toutes `verifiee` et primaires : 10 `confirme`, 3 `contextualise` (Sénat et Assemblée nationale)

1.4 Les deux vues destinées au site

Une **vue** est une requête enregistrée, que l'on interroge comme une table. Nos deux vues sont créées avec `security_invoker = true` : elles appliquent la RLS **de la personne qui lit**. Elles ne peuvent donc jamais montrer plus que les tables elles-mêmes.

`v_feed_verifie` — le fil des articles.

- **Ce qu'elle contient** : une ligne par article réel visible (`where c.visible_public and not c.est_test`).
- **Ses colonnes** : `titre`, `media_nom`, `url_originale`, `domaine_origine`, `theme`, `date_publication`, plus des compteurs calculés (`nb_analyses_publiees`, `nb_abstentions`, `statut_verification` et `derniere_analyse_le`).
- **Le statut de vérification** vaut `analyse_disponible`, `verification_insuffisante` ou `non_verifie`.

`v_analyses_publiques` — la fiche d'une analyse.

- **Ce qu'elle contient** : seulement les affirmations en `analyse_disponible` ou `verification_insuffisante`, d'un article visible et non fictif.
- **Ses colonnes** :
  - le texte de l'affirmation, le verdict, l'explication, les limites et le motif d'abstention ;
  - la mention fixe **« Analyse produite par IA »** ;
  - `revue_par_equipe` ;
  - les informations de l'article ;
  - un champ `preuves` (JSON), qui ne regroupe que les preuves **vérifiées** de la **version en cours**.

Pourquoi le site doit passer par ces vues :

1. **Une seule requête** pour afficher un fil ou une fiche complète.
2. **Les filtres publics sont déjà appliqués** : ni fictif, ni article masqué, ni analyse interne.
3. **Aucun champ interne** : pas d'`historique`, de `motif_sensibilite` ni d'empreintes.
4. **La mention IA et les limites** sont toujours fournies.
5. **La RLS reste appliquée**, grâce à `security_invoker`.

2. Rôles et utilisateurs

2.1 Un utilisateur Supabase Auth n'est pas un rôle PostgreSQL

	Utilisateur Supabase Auth	Rôle PostgreSQL
Ce que c'est	Une **personne**, enregistrée comme une ligne de `auth.users` (e-mail, mot de passe)	Une **identité technique** de la base, avec ses droits (`GRANT`)
Exemples	`admin@demelo.test` , `visiteur@demelo.test` , les comptes de l'équipe	`anon` , `authenticated` , `service_role` , `demelo_pipeline`
Lien entre les deux	À la connexion, Supabase délivre un **jeton JWT**. L'API exécute alors la requête avec le rôle `authenticated` .	Sans jeton (visiteur), la requête s'exécute avec le rôle `anon` .

À retenir : tous les utilisateurs connectés partagent le **même** rôle PostgreSQL, `authenticated` . Ce qui distingue un administrateur, c'est une information **à l'intérieur de son jeton**, et non un rôle PostgreSQL différent.

2.2 Les profils réellement configurés

Profil	Comment on le reconnaît	Ce qu'il fait dans Démêlo
`anon` — visiteur	Aucune connexion, seulement la clé publique	Lit ce qui est public
`authenticated` — connecté	Il possède un JWT valide	Rôle de base de tous les comptes
Connecté sans privilège	`authenticated` , sans `role_equipe`	Mêmes droits qu'un visiteur
Administrateur	`authenticated` + `app_metadata.role_equipe = 'admin'`	Voit tout et écrit, toujours via les triggers
`demelo_pipeline`	Rôle technique, connexion par mot de passe (secret GitHub)	Collecte RSS et écriture des analyses, **sans suppression**
`service_role`	Clé **secrète** de Supabase, avec BYPASSRLS	Réservé à l'administration de Supabase. **Jamais** dans le site ni dans le dépôt. Dans notre projet, il n'a même pas le droit `UPDATE` sur nos tables (test W4).

Les comptes de test, créés dans *Authentication > Users* (mots de passe non communiqués) :

- `admin@demelo.test` porte `role_equipe = admin` : il sert à tester les droits d'un administrateur ;
- `visiteur@demelo.test` n'a **aucun rôle** : il sert à tester un compte connecté sans privilège.

Les comptes réels de l'équipe reçoivent le rôle admin de la même manière.

`demelo_pipeline` **, un rôle restreint par construction** :

- ni `SUPERUSER` , ni `BYPASSRLS` , ni `CREATEROLE` , et `NOINHERIT` ;
- aucun accès au schéma `auth` ni à `archive_v1` ;
- sur `contenus` , des droits limités **colonne par colonne** ;
- **aucun** `DELETE` .

2.3 Comment reconnaître un administrateur : `prive.est_admin()`

```
create function prive.jwt() returns jsonb language sql stable set search_path = '' as $$
  select nullif(current_setting('request.jwt.claims', true), '')::jsonb;
$$;

create or replace function prive.role_equipe() returns text language sql stable set search_path = '' as
$$
  select case
           when coalesce((prive.jwt() ->> 'is_anonymous')::boolean, false) then null
           else prive.jwt() -> 'app_metadata' ->> 'role_equipe'
         end;
$$;

create function prive.est_admin() returns boolean language sql stable set search_path = '' as $$
  select coalesce(prive.role_equipe() = 'admin', false);
$$;
```

1. **`prive.jwt()`** lit le jeton de la requête en cours, que Supabase place dans `request.jwt.claims`.
2. **`prive.role_equipe()`** renvoie `app_metadata.role_equipe`, **sauf** pour un compte anonyme, qui n'est jamais admin (test W7).
3. **`prive.est_admin()`** renvoie `true` seulement si ce rôle vaut `'admin'`, et `false` dans tous les autres cas, y compris en l'absence de jeton.
4. Ces fonctions sont dans le schéma **`prive`**, que l'API n'expose pas.

Pourquoi `app_metadata` et pas `user_metadata` ? Un utilisateur peut modifier **son propre** `user_metadata` depuis le navigateur. S'il suffisait d'y écrire `role_equipe = admin`, n'importe qui deviendrait administrateur. Le `app_metadata`, lui, ne se modifie que côté serveur, par nous, dans le SQL Editor :

```
update auth.users
   set raw_app_meta_data = coalesce(raw_app_meta_data, '{}'::jsonb) || '{"role_equipe": "admin"}'
 where email = '<compte à promouvoir>' and email_confirmed_at is not null;
```

2.4 Qui a le droit de faire quoi ?

Action	Visiteur `anon`	Connecté sans rôle	Admin	`demelo_pipeline`
Lire les 15 sources actives	✅	✅	✅ (et les inactives)	✅ (actives)
Lire les articles réels visibles	✅ (260)	✅ (260)	✅ (760, fictifs compris)	✅ (réels seulement)
Voir les 500 données fictives	❌	❌	✅	❌
Lire les analyses publiées	✅ (9)	✅ (9)	✅ (10)	✅
Voir une analyse en cours ou en revue	❌	❌	✅	✅
Lire les preuves vérifiées publiées	✅ (13)	✅ (13)	✅ (toutes)	✅

Action	Visiteur `anon`	Connecté sans rôle	Admin	`demelo_pipeline`
Ajouter ou modifier une ligne	❌ (pas de GRANT)	❌ (policy)	✅ (contrôlé par les triggers)	✅ (colonnes autorisées, triggers)
Supprimer une ligne	❌	❌	❌ (pas de GRANT)	❌ (pas de GRANT)

3. La sécurité RLS en profondeur

3.1 Les bases

La Row Level Security (RLS) est un filtre que PostgreSQL applique **ligne par ligne**. Quand la RLS est activée sur une table, une ligne n'est visible ou modifiable que si une **policy** l'autorise. **Sans aucune policy, tout est refusé.**

Pourquoi l'activer sur les 4 tables ?

- La clé du site est **publique** : n'importe qui peut interroger l'API.
- La RLS garantit qu'avec cette clé on n'obtient **que les lignes publiques**, quel que soit le code du site.
- Elle est activée dès la création des tables (migration `v2_02`). La base est donc restée **fermée** jusqu'à ce que nous écrivions nos règles.

Les deux portes successives :

Porte	Question posée	Si elle refuse
`GRANT` (privilège)	Ce **rôle** a-t-il le droit de tenter cette **action** sur cette table ?	Erreur `permission denied for table …`
`POLICY` (RLS)	Sur **quelles lignes** cette action est-elle permise ?	En lecture : 0 ligne, sans erreur. En écriture : `new row violates row-level security policy`, ou 0 ligne modifiée.

Il faut **les deux** : une policy sans GRANT n'ouvre rien, et un GRANT sans policy ne montre aucune ligne.

`USING` et `WITH CHECK` :

Opération	`USING`	`WITH CHECK`
`SELECT` (lire)	✅ quelles lignes existantes je **vois**	—
`INSERT` (ajouter)	—	✅ la **nouvelle** ligne est-elle acceptée ?
`UPDATE` (modifier)	✅ quelles lignes je peux **viser**	✅ à quoi la ligne peut ressembler **après**
`DELETE` (supprimer)	✅ quelles lignes je peux supprimer	—

Comment PostgreSQL décide, pour chaque requête :

1. **Le rôle** : l'API choisit `anon` ou `authenticated` d'après le jeton.
2. **Le GRANT** : sans privilège sur l'action, la requête est refusée immédiatement.
3. **Les policies** : PostgreSQL ajoute automatiquement la condition `USING` à la requête. Si plusieurs policies visent la même opération, elles s'additionnent (**OU**).
4. `WITH CHECK` : en écriture, la nouvelle ligne est vérifiée.
5. **Les triggers** (`BEFORE`) et les **contraintes** vérifient enfin les règles métier.

Une astuce de performance : nous écrivons `(select prive.est_admin())`, entre parenthèses. Ainsi, la fonction est calculée **une seule fois par requête**, et non pour chaque ligne.

3.2 Les 12 policies que nous avons écrites nous-mêmes

Elles sont tirées de `supabase/rls_manuel/policies_et_grants.sql` et ont été saisies **à la main** par l'équipe, en suivant `docs/tutoriel-rls.md`. Toutes visent `anon` et/ou `authenticated`.

Table `sources`

P1 — `sources_lecture` (SELECT, `anon` et `authenticated`)

```
create policy sources_lecture on public.sources
  for select to anon, authenticated
  using (active or (select prive.est_admin()));
```

- Tout le monde voit les organismes **actifs**. Un organisme désactivé n'est visible que par un admin.
- *Exemple* : un visiteur voit les 15 organismes, mais pas un organisme de test désactivé.

P2 — `sources_ajout` (INSERT, `authenticated`)

```
create policy sources_ajout on public.sources
  for insert to authenticated
  with check ((select prive.est_admin()));
```

- Seul un admin peut ajouter un organisme.
- *Exemple* : pour `visiteur@demelo.test`, l'ajout est refusé (`new row violates row-level security policy`).

P3 — `sources_modif` (UPDATE, `authenticated`)

```
create policy sources_modif on public.sources
  for update to authenticated
  using ((select prive.est_admin()))
  with check ((select prive.est_admin()));
```

- `USING` : seul un admin peut viser une ligne. `WITH CHECK` : la ligne modifiée doit rester acceptable.
- *Exemple* : un compte sans rôle « modifie » 0 ligne, sans erreur, parce que la ligne lui est invisible pour l'`UPDATE`.

Table `contenus`

P4 — `contenus_lecture` (SELECT, `anon` et `authenticated`)

```
create policy contenus_lecture on public.contenus
  for select to anon, authenticated
  using ((visible_public and not est_test) or (select prive.est_admin()));
```

- **La policy qui protège les 500 lignes fictives.** Le public ne voit que les articles visibles **et** non fictifs. L'admin voit tout.
- *Exemple* : un visiteur voit 260 articles, et un admin en voit 760.

P5 — `contenus_ajout` (INSERT, `authenticated`) et **P6 — `contenus_modif`** (UPDATE, `authenticated`)

```
create policy contenus_ajout on public.contenus
  for insert to authenticated
  with check ((select prive.est_admin()));

create policy contenus_modif on public.contenus
  for update to authenticated
  using ((select prive.est_admin()))
  with check ((select prive.est_admin()));
```

- Seul un admin peut indexer ou corriger un article : par exemple ajouter une donnée fictive, ou masquer un article.

Table `affirmations` ⚠ (elle contient les analyses non publiées)

P7 — `affirmations_lecture` (SELECT, `anon` et `authenticated`)

```
create policy affirmations_lecture on public.affirmations
  for select to anon, authenticated
  using (
    (statut_analyse in ('analyse_disponible', 'verification_insuffisante')
     and exists (select 1 from public.contenus c where c.id = contenu_id))
    or (select prive.est_admin())
  );
```

- **Ligne 1** : seules les analyses **publiées** ou les **abstentions** sont publiques.
- **Ligne 2** : la sous-requête sur `contenus` **passe elle-même par P4**. Une analyse d'un article masqué ou fictif reste donc invisible.
- **Ligne 3** : l'admin voit tout.
- *Exemple* : un visiteur voit les 9 analyses publiées, mais pas l'analyse n° 68 en `revue_requise`.

P8 — `affirmations_ajout` (INSERT) et **P9 — `affirmations_modif`** (UPDATE), pour `authenticated`

```
create policy affirmations_ajout on public.affirmations
  for insert to authenticated
  with check ((select prive.est_admin()));

create policy affirmations_modif on public.affirmations
  for update to authenticated
  using ((select prive.est_admin()))
  with check ((select prive.est_admin()));
```

- Seul un admin peut créer ou modifier une affirmation. **Ce qui est permis** (transitions, preuves exigées, motif) est ensuite décidé par le trigger `controler_affirmation`.

Table `preuves` ⚠ (elle révèle les analyses non publiées et les échecs)

P10 — `preuves_lecture` (SELECT, `anon` et `authenticated`)

```
create policy preuves_lecture on public.preuves
  for select to anon, authenticated
  using (
    (statut_preuve = 'verifiee'
     and exists (select 1 from public.affirmations a
                 where a.id = affirmation_id and a.version = version_analyse))
    or (select prive.est_admin())
  );
```

- Le public ne voit que les preuves **vérifiées**, de la **version en cours** d'une affirmation **qu'il a le droit de voir** : la sous-requête passe par P7.
- *Exemple* : un échec (`inaccessible`, `rejetee`) ou la preuve d'une analyse en cours restent invisibles.

P11 — `preuves_ajout` (INSERT) et **P12 — `preuves_modif`** (UPDATE), pour `authenticated`

```
create policy preuves_ajout on public.preuves
  for insert to authenticated
  with check ((select prive.est_admin()));

create policy preuves_modif on public.preuves
  for update to authenticated
  using ((select prive.est_admin()))
  with check ((select prive.est_admin()));
```

- Seul un admin peut ajouter ou corriger une preuve. Le trigger `controler_preuve` ajoute ses propres conditions : uniquement pendant l'analyse, dans le périmètre de l'organisme.

3.3 Les 5 groupes de GRANT (écrits à la main)

```
-- G1 sources
grant select on public.sources to anon, authenticated;
grant insert, update on public.sources to authenticated;
-- G2 contenus
grant select on public.contenus to anon, authenticated;
grant insert, update on public.contenus to authenticated;
-- G3 affirmations
grant select on public.affirmations to anon, authenticated;
grant insert, update on public.affirmations to authenticated;
-- G4 preuves
grant select on public.preuves to anon, authenticated;
grant insert, update on public.preuves to authenticated;
-- G5 vues
grant select on public.v_feed_verifie, public.v_analyses_publiques to anon, authenticated;
```

Pourquoi ils sont nécessaires :

- **Sans GRANT, la policy ne sert à rien** : PostgreSQL refuse avant même de regarder les lignes. La migration avait volontairement **retiré** tous les droits clients.
- **`anon` ne reçoit que `SELECT`** : un visiteur ne peut écrire nulle part. Le refus vient du GRANT (`permission denied`).
- **Aucun `DELETE`, pour personne** : on **retire** une analyse, on ne la supprime pas, et les échecs restent tracés.

- **G5 suffit pour les vues** : avec `security_invoker`, elles appliquent les policies des tables, donc aucune policy supplémentaire n'est nécessaire.
- **Ordre suivi pour chaque table** : policies, puis GRANT, puis tests. Ainsi, aucune table n'a jamais été ouverte sans ses règles.

3.4 Les 6 policies du pipeline (déjà présentes, créées par la migration `v2_05`)

Nous ne les avons pas écrites à la main : elles font partie de l'architecture, et ne concernent que le rôle serveur `demelo_pipeline`.

Policy	Table	Opération	Condition
`pipeline_sources_lecture`	sources	SELECT	`using (active)`
`pipeline_contenus_lecture`	contenus	SELECT	`using (not est_test)`
`pipeline_contenus_ajout`	contenus	INSERT	`with check (not est_test and source_id is null)`
`pipeline_contenus_suivi`	contenus	UPDATE	`using (not est_test) with check (not est_test)`
`pipeline_affirmations`	affirmations	ALL	`using (true) with check (true)` — le trigger contrôle tout
`pipeline_preuves`	preuves	ALL	`using (true) with check (true)` — le trigger contrôle tout

À dire clairement :

- Le pipeline **ne voit jamais** les données fictives.
- Le `using (true)` est acceptable ici, pour trois raisons : ce rôle n'est pas accessible depuis l'API, il n'a **aucun droit** `DELETE`, et le trigger refuse toute transition interdite.
- **Total : 12 policies manuelles + 6 policies du pipeline = 18 policies.**

4. Les autres mécanismes SQL

4.1 Les clés étrangères

Une **clé étrangère** oblige une colonne à pointer vers une ligne qui **existe** dans une autre table. Par exemple, `affirmations.contenu_id` doit correspondre à un `contenus.id` réel.

- `ON DELETE RESTRICT` (presque partout) : impossible de supprimer un article qui a des affirmations, ou un organisme cité dans des preuves.
- `ON DELETE CASCADE` (`preuves.affirmation_id`) : si une affirmation disparaît, ses preuves disparaissent avec elle.
- **Bilan vérifié** : 8 clés étrangères, toutes validées, 0 ligne orpheline.

4.2 Les contraintes d'intégrité (indépendantes des RLS)

Contrainte	Ce qu'elle garantit
`UNIQUE (url_originale)`, `UNIQUE (domaine_origine, identifiant_externe)`	Pas de doublon d'article
`contenus_fictif_check`	Une ligne **fictive** a forcément un domaine `.invalid`, un titre `[FICTIF] …`, un média « Média fictif… » et aucun organisme. Une ligne **réelle** ne peut **jamais** avoir un domaine `.invalid`.
`affirmations_publication_check`	Pour publier, il faut un verdict, une explication, des limites, la citation vérifiée, le modèle, la version des consignes, la date d'analyse et la date de publication.
`affirmations_abstention_check`	Une abstention a un motif et des limites, et pas de verdict.
`affirmations_nature_verdict_check`	`non_verifiable` est réservé aux opinions, promesses et prévisions.
`affirmations_recoupement_check`	Un verdict vrai, faux ou trompeur exige un recoupement complet : période, définition, unité, population, périmètre, contradictions et dépendances.
`affirmations_sensible_check` / `affirmations_revue_check`	Un cas sensible a un motif, et une revue a une raison.
`preuves_verifiee_check`	Une preuve vérifiée a une URL finale en https, un HTTP 2xx, une empreinte, un titre, une nature, un passage retrouvé, un domaine conforme, un extrait ou une empreinte du passage, et une date ou une version.
`sources_perimetres_check`	Chaque organisme a au moins un domaine, écrit sans `https://` ni `www.`

4.3 Les fonctions SQL (schéma `prive`, non exposé par l'API)

- **Identité** : `jwt()`, `uid()`, `role_equipe()`, `est_admin()`.
- **Contrôle** : `url_dans_perimetre(url, perimetres)` vérifie qu'une URL est en https et dans le domaine ou le chemin de l'organisme (sous-domaines inclus).
- **Lectures utiles aux triggers**, en `SECURITY DEFINER` : `contenu_est_test`, `etat_affirmation`, `infos_source`, `preuve_de_la_meme_analyse` et `bilan_preuves`, qui compte les preuves vérifiées, primaires, qui confirment, qui contredisent. Elles lisent les données **quelles que soient les policies de l'appelant**, avec un `search_path` vide pour éviter tout détournement.

- **Utilitaire** : `maj_updated_at()` met à jour automatiquement la date de modification.

4.4 Les triggers qui encadrent la publication

`controler_affirmation` (`BEFORE INSERT OR UPDATE` sur `affirmations`) :

1. **Qui peut écrire ?** Seulement le pipeline ou un admin. Tout autre appelant est refusé, même avec la clé `service_role`.
2. **Création** : toujours en `non_traite`, et jamais sur un contenu fictif.
3. **Transitions autorisées** :
   - `non_traite` → `en_cours` ;
   - `en_cours` → `analyse_disponible`, `verification_insuffisante`, `revue_requise` ou retour à `non_traite` ;
   - un admin fait passer `revue_requise` → publié, abstention ou retiré ;
   - un admin peut retirer une analyse publiée, ou la renvoyer en `non_traite` (la version augmente de 1).
4. **Motif obligatoire** (`revue_motif`) pour chaque décision d'admin. Ce motif est archivé dans `historique`.
5. **Analyse figée** : une fois publiée, une analyse ne peut plus être réécrite. Il faut la renvoyer en `non_traite`.
6. **Règles de publication** :
   - **vrai** : au moins une preuve **primaire** vérifiée qui confirme, et **aucune** qui contredit ;
   - **faux** : au moins une preuve primaire qui contredit, et aucune qui confirme ;
   - **trompeur** : une preuve qui confirme, une qui contextualise ou contredit, et au moins une primaire ;
   - un **cas sensible** ne peut être publié **qu'après** être passé par `revue_requise`, sur décision d'un admin.

`controler_preuve` (`BEFORE INSERT, UPDATE OR DELETE` sur `preuves`) :

- **Quand** : une preuve n'est acceptée **que pendant l'analyse** (`en_cours`) ; ensuite, elle est figée.
- **Où** : l'URL doit appartenir au **périmètre** de l'organisme cité, sinon la preuve est refusée.
- **Licence** : pas d'**extrait** pour une source en `lien_seulement`, seulement une empreinte du passage.
- **Traçabilité** : `version_analyse` et `ajoutee_par` sont imposés par la base.

Pourquoi des triggers en plus des RLS ? La RLS décide **qui voit et qui modifie quelles lignes**. Les triggers décident **si une analyse a le droit d'être publiée**, et ils s'appliquent à **tous les rôles**. Même un admin ne peut pas publier un « vrai » sans preuve primaire (test W3).

4.5 La protection des 500 données fictives

L'objectif était d'atteindre au moins 500 lignes **sans fausse information publique**. La protection repose sur **5 verrous** :

1. `est_test = true` et la contrainte `contenus_fictif_check` : une ligne fictive est reconnaissable par construction (domaine `.invalid`, titre `[FICTIF] `).
2. **La policy P4** : `not est_test` pour le public.
3. **Les policies du pipeline** : `not est_test`, donc le pipeline ne les voit pas et ne les analyse jamais.
4. **Le trigger** : aucune affirmation n'est possible sur un contenu fictif (test W6).
5. **Les vues** : filtre `not c.est_test`.

4.6 Une affirmation sensible en `revue_requise`

- **Ce que c'est** : l'affirmation **n° 68**, la déclaration ministérielle selon laquelle « les deux tiers » des contenus liant les violences du 17 octobre à l'immigration viendraient de l'étranger.
- **Pourquoi elle est sensible** : le sujet est sensible (`sensible = true`, avec un motif), et aucune publication officielle ne donne ce chiffre (motif d'abstention renseigné).

- **Où elle en est** : elle est en `revue_requise`, **invisible du public**. Un admin doit trancher, avec un motif : publier en abstention, ou écarter.

5. Tests et validation

5.1 Les 52 tests SQL (`supabase/rls_manuel/tests_sql_editor.sql`)

Comment ça marche :

- Le script s'exécute dans le **SQL Editor de Supabase**, avec les **vrais rôles** (`anon`, `authenticated`, `service_role`) et un jeton simulé pour chaque compte de test.
- Il crée des données de test, lance chaque essai, puis **annule tout** : rien ne reste en base.
- Il affiche ✅ (conforme), ❌ (non conforme) ou ⏳ (table pas encore ouverte).
- **Résultat final : 52 ✅ / 0 ❌ / 0 ⏳.**

Famille	Nombre	Ce qu'elle vérifie
P1 à P12	35	Chaque policy : ce qui est autorisé **et** ce qui est refusé
G1 à G5	10	Les GRANT : pas d'écriture pour `anon`, pas de suppression, les vues
W1 à W7	7	Les règles de publication des triggers

Les 7 tests W :

- **W1** : une décision d'admin sans motif est bloquée.
- **W2** : la revue d'un cas sensible avec motif est acceptée.
- **W3** : un « vrai » sans preuve primaire est bloqué.
- **W4** : la clé `service_role` ne peut pas modifier une analyse.
- **W5** : une analyse publiée ne peut pas être réécrite.
- **W6** : une affirmation sur un contenu fictif est bloquée.
- **W7** : un compte anonyme portant un rôle admin forgé est bloqué.

5.2 L'histoire du test W4 : 51/52, puis 52/52

- **Le symptôme** : sur Supabase, le test affichait ⏳ au lieu de ✅.
- **La cause** : sur notre projet, `service_role` n'a **pas le droit** `UPDATE` sur nos tables. Il est donc refusé dès le GRANT (`permission denied`), **avant** même le trigger. Le script classait ce refus en ⏳, c'est-à-dire « table pas encore ouverte ».
- **Le diagnostic** : nous avons rejoué l'essai sur le vrai projet, et le refus a été confirmé. La protection est donc **plus stricte** que prévu.
- **La correction** : uniquement **dans le script de test**. W4 accepte désormais « bloqué par le trigger **ou** par l'absence de droit » (`bloque_ou_refus`).
- **Ce qui n'a pas changé** : aucune policy, aucun GRANT, aucun trigger.

5.3 Les vérifications des profils (sur le vrai projet)

Contrôle	Résultat
Visiteur `anon`	260 articles, 0 fictif, 9 analyses, 13 preuves. Écriture refusée (`permission denied`).
Compte connecté sans rôle	260 articles, 0 fictif, 9 analyses. Une tentative de modification touche 0 ligne.
Administrateur	760 contenus (dont 500 fictifs), 10 affirmations (dont l'analyse n° 68 en revue), 13 preuves

Contrôle	Résultat
Données fictives publiques	**0**, ni dans `contenus`, ni dans `v_feed_verifie`
Analyse non publiée	l'analyse n° 68 est **invisible** pour le visiteur et pour le compte sans rôle
Intégrité	8 clés étrangères validées, 0 orphelin, 18 policies, aucun `DELETE` accordé aux clients

En plus, nos **tests locaux automatiques** rejouent toute la base dans un PostgreSQL jetable (74 contrôles réussis). Ils vérifient aussi la base fermée et les règles du pipeline.

5.4 Ce qui reste à faire (à dire honnêtement)

- **Les 18 tests automatisés par l'API Supabase n'ont pas encore été exécutés.** Le script `tests_api.mjs` et le workflow GitHub *Tests API Supabase* sont prêts. Ils attendent que nous créions les **5 secrets GitHub** : la clé publique et les comptes de test. Il faudra ensuite supprimer les 3 lignes de test qu'ils créent (`nettoyage_tests_api.sql`).
- Les **connexions anonymes** doivent encore être désactivées dans Supabase Auth.

6. Questions possibles du professeur

1. Pourquoi Supabase plutôt qu'un simple fichier ? Un fichier n'a ni relations, ni droits d'accès, ni authentification. Supabase nous donne PostgreSQL (clés, contraintes, triggers), Auth et une API sécurisée par RLS.

2. Pourquoi quatre tables ? Une table par notion : les **organismes** (sources), les **articles** (contenus), les **phrases vérifiées** (affirmations) et les **documents** (preuves). Chaque information n'est stockée qu'une fois.

3. Qu'est-ce qu'une clé étrangère ? Une colonne qui doit pointer vers une ligne existante d'une autre table. Par exemple, `affirmations.contenu_id` pointe vers `contenus.id`. Impossible donc d'avoir une affirmation sans article.

4. Qu'est-ce qu'une RLS ? Un filtre ligne par ligne, appliqué par PostgreSQL lui-même. Une ligne n'est visible ou modifiable que si une policy l'autorise ; sans policy, tout est refusé.

5. Quelle différence entre une policy et un GRANT ? Le GRANT dit **quelle action** un rôle peut tenter (lire, ajouter…). La policy dit **sur quelles lignes**. Il faut les deux.

6. Et entre `USING` et `WITH CHECK` ? `USING` filtre les lignes **existantes**, celles qu'on voit ou qu'on vise. `WITH CHECK` valide la ligne **nouvelle ou modifiée**.

7. Comment reconnaissez-vous un admin ? Grâce à `prive.est_admin()`, qui lit `app_metadata.role_equipe = 'admin'` dans le jeton de connexion. Nous utilisons `app_metadata` parce que l'utilisateur ne peut pas le modifier lui-même, contrairement à `user_metadata`.

8. Pourquoi 500 lignes fictives ? Le cahier des charges demandait au moins 500 lignes. Nous avons 260 vrais articles, et nous avons complété avec 500 lignes **clairement marquées** (`est_test`, domaine `.invalid`). Elles sont invisibles du public et ne sont jamais analysées.

9. Comment savez-vous qu'un visiteur ne peut rien modifier ? `anon` n'a que le droit `SELECT`, donc toute écriture est refusée par le GRANT. Les tests G1 à G4 le prouvent, et nous l'avons vérifié sur le vrai projet.

10. Pourquoi des triggers en plus des RLS ? La RLS décide **qui** accède à **quelles lignes**. Les triggers vérifient les **règles métier** : un « vrai » exige une preuve primaire, un cas sensible passe par une revue, un motif est obligatoire. Ils s'appliquent à tout le monde, y compris aux admins.

11. Comment le site récupère-t-il les données ? Par l'API Supabase, avec la **clé publique** uniquement, en lisant les vues `v_feed_verifie` et `v_analyses_publiques`. Comme ces vues sont en `security_invoker`, la RLS du visiteur s'applique.

12. Que se passerait-il si la clé publique fuitait ? Rien de grave : elle est publique par nature. La sécurité repose sur la RLS et les GRANT. Seule la clé `service_role` est secrète, et elle n'est ni dans le site, ni dans le dépôt.

Annexe — Le SQL réel

A. Nos 12 policies et nos 5 GRANT (`supabase/rls_manuel/policies_et_grants.sql`)

```
-- TABLE 1 — sources
create policy sources_lecture on public.sources
  for select to anon, authenticated
  using (active or (select prive.est_admin()));

create policy sources_ajout on public.sources
  for insert to authenticated
  with check ((select prive.est_admin()));

create policy sources_modif on public.sources
  for update to authenticated
  using ((select prive.est_admin()))
  with check ((select prive.est_admin()));

grant select on public.sources to anon, authenticated;
grant insert, update on public.sources to authenticated;

-- TABLE 2 — contenus
create policy contenus_lecture on public.contenus
  for select to anon, authenticated
  using ((visible_public and not est_test) or (select prive.est_admin()));

create policy contenus_ajout on public.contenus
  for insert to authenticated
  with check ((select prive.est_admin()));

create policy contenus_modif on public.contenus
  for update to authenticated
  using ((select prive.est_admin()))
  with check ((select prive.est_admin()));

grant select on public.contenus to anon, authenticated;
grant insert, update on public.contenus to authenticated;

-- TABLE 3 — affirmations
create policy affirmations_lecture on public.affirmations
  for select to anon, authenticated
  using (
    (statut_analyse in ('analyse_disponible', 'verification_insuffisante')
     and exists (select 1 from public.contenus c where c.id = contenu_id))
    or (select prive.est_admin())
  );

create policy affirmations_ajout on public.affirmations
  for insert to authenticated
  with check ((select prive.est_admin()));
```

```
create policy affirmations_modif on public.affirmations
  for update to authenticated
  using ((select prive.est_admin()))
  with check ((select prive.est_admin()));

grant select on public.affirmations to anon, authenticated;
grant insert, update on public.affirmations to authenticated;

-- TABLE 4 — preuves
create policy preuves_lecture on public.preuves
  for select to anon, authenticated
  using (
    (statut_preuve = 'verifiee'
     and exists (select 1 from public.affirmations a
                 where a.id = affirmation_id and a.version = version_analyse))
    or (select prive.est_admin())
  );

create policy preuves_ajout on public.preuves
  for insert to authenticated
  with check ((select prive.est_admin()));

create policy preuves_modif on public.preuves
  for update to authenticated
  using ((select prive.est_admin()))
  with check ((select prive.est_admin()));

grant select on public.preuves to anon, authenticated;
grant insert, update on public.preuves to authenticated;

-- VUES
grant select on public.v_feed_verifie, public.v_analyses_publiques to anon, authenticated;
```

B. Les 6 policies et les droits du pipeline (migration `20261028165254_v2_05_pipeline_ia.sql`)

```
grant usage on schema public, prive to demelo_pipeline;
grant select on public.sources, public.contenus, public.affirmations, public.preuves to
demelo_pipeline;
grant insert (media_nom, url_originale, titre, identifiant_externe, date_publication, theme,
lot_import, vu_le)
  on public.contenus to demelo_pipeline;
grant update (statut_traitement, derniere_analyse_le, derniere_erreur, vu_le) on public.contenus to
demelo_pipeline;
grant insert, update on public.affirmations, public.preuves to demelo_pipeline;

create policy pipeline_sources_lecture on public.sources
  for select to demelo_pipeline using (active);
create policy pipeline_contenus_lecture on public.contenus
  for select to demelo_pipeline using (not est_test);
create policy pipeline_contenus_ajout on public.contenus
  for insert to demelo_pipeline with check (not est_test and source_id is null);
create policy pipeline_contenus_suivi on public.contenus
  for update to demelo_pipeline using (not est_test) with check (not est_test);
create policy pipeline_affirmations on public.affirmations
  for all to demelo_pipeline using (true) with check (true);   -- le trigger 8.1 contrôle tout
create policy pipeline_preuves on public.preuves
  for all to demelo_pipeline using (true) with check (true);   -- le trigger 8.2 contrôle tout
```

C. Les migrations, dans l'ordre (`supabase/migrations/`)

Migration	Rôle
`v2_01_archiver_v1`	Archive l'ancienne version dans le schéma `archive_v1`, non exposé par l'API. Rien n'est supprimé.
`v2_02_schema_cible`	Crée les 4 tables, avec la RLS activée **dès la création** et aucun droit client.
`v2_03_donnees_initiales`	Ajoute les sources de départ.
`v2_04_sources_retenues`	Ne garde que les **15 organismes** retenus par l'équipe.
`v2_05_pipeline_ia`	Ajoute les statuts d'analyse, les colonnes de preuve, les fonctions `prive`, les triggers, les 2 vues, le rôle `demelo_pipeline` et ses 6 policies.

Fiche rédigée à partir du code du dépôt et du schéma réel de Supabase, vérifiés le 28 octobre 2026.